Event-Driven Systems in Practice

Notes on Kafka, Temporal, DynamoDB and Datadog for Node.js teams

October 2026

Contents

About this document . . . 3

1. Why events at all . . . 3

2. The vocabulary . . . 4

3. Choreography and orchestration . . . 6

4. Living with at-least-once delivery . . . 7

5. Temporal: durable workflows . . . 8

6. DynamoDB as the system of record . . . 11

7. Seeing what's happening: Datadog . . . 14

8. Putting it together . . . 17

9. Running it . . . 19

10. Before going live . . . 20

Further reading . . . 20

About this document

This is the document I wish someone had handed me before our first event-driven project. It isn't a reference manual and it doesn't try to cover every option of every tool. It's an attempt to explain how the pieces fit together, why you would choose them, and where they tend to hurt once real traffic arrives.

The examples come from a small order platform: customers place orders, stock gets reserved, a payment service charges the card, and the customer gets an e-mail. It's a deliberately ordinary domain. The interesting parts are not the business rules but what happens between the services: messages that arrive twice, services that are down for twenty minutes, a payment that succeeds after we already gave up on it.

The stack is the one many Node.js teams on AWS end up with. Kafka carries the events. Temporal runs the long-lived business processes. DynamoDB holds the state. Datadog is how we find out what's going on. The code is TypeScript, written for Node 24, and kept short on purpose. If you want something you can run, the companion repository *eda-commerce* has a working version of the order flow with tests against a real Kafka broker and a real Temporal server.

You don't need to read this front to back. The first three sections are about event-driven design in general and apply whatever tools you use. The middle sections take one tool each. The last part puts everything together and talks about running it.

1. Why events at all

Most systems start life as a set of services that call each other. The order service calls inventory, then payment, then the e-mail service, and returns a response once all of them have answered. That works, and for a long time it works well. The trouble shows up gradually.

The first sign is usually availability. If the order service needs four other services to be healthy before it can accept an order, its availability is roughly the product of theirs. Four services at 99.9% each give you something closer to 99.6%, and that's before network problems. The second sign is coupling of a more human kind. Every new feature that needs to react to an order (loyalty points, fraud scoring, a warehouse feed) means another call added to the order service, owned by a team that has no particular interest in loyalty points.

Event-driven architecture flips the direction of knowledge. Instead of the order service knowing who needs to hear about an order, it announces that an order was placed, and anyone who cares listens. The order service doesn't wait for them and doesn't know they exist. A new consumer can be added on a Tuesday afternoon without anyone touching the producer.

That freedom has a price, and it's worth being honest about it up front:

- **You give up immediate consistency.** When the API answers, the order exists, but the stock isn't reserved yet and the payment hasn't happened. The client has to live with "accepted, in progress".
- **Failures become quieter.** A synchronous call that fails gives you a stack trace. An event that nobody processed gives you nothing until a customer asks where their order is.

– **Flows become harder to see.** The business process "place an order" no longer lives in one function. It is spread across services that react to each other.

Most of this document is about paying that price sensibly: keeping the decoupling while getting back some of the visibility and the guarantees.

> *A useful rule of thumb: if the caller genuinely needs the answer to continue (checking a password, quoting a price), a synchronous call is fine and usually simpler. Events earn their keep when the caller doesn't need to wait, or when several parties need to react to the same fact.*

2. The vocabulary

A lot of confusion in event-driven projects comes from people using the same words for different things. It helps to agree on a few terms early.

2.1 Events and commands

An **event** is a statement about something that has already happened: `OrderPlaced`, `PaymentSucceeded`, `StockReserved`. It's named in the past tense, it can't be rejected (it already happened), and the producer doesn't care how many consumers it has, including zero.

A **command** is a request for someone to do something: `RequestPayment`, `ReserveStock`. It has exactly one intended receiver, it can fail, and the sender usually expects to hear back.

Both can travel over Kafka, and in our platform they do. The difference matters because it tells you who owns the contract. An event's schema belongs to its producer; consumers adapt to it. A command's schema belongs to the receiver, since it defines the receiver's API. Mixing the two up leads to "events" like `SendWelcomeEmail` that are really commands in disguise, and to producers that quietly start depending on a particular consumer.

2.2 Topics, partitions and consumer groups

Kafka stores messages in **topics**, and each topic is split into **partitions**. A partition is an ordered, append-only log. Kafka guarantees order *within* a partition and nothing more. When you publish a message with a key, Kafka hashes the key to pick the partition, so all messages with the same key land in the same partition, in the order they were written.

This is the single most important design decision you make for a topic. If all events for one order must be processed in order, use the order ID as the key. Then `OrderPlaced` is guaranteed to be read before `OrderCancelled` for the same order, while different orders are spread across partitions and processed in parallel.

A **consumer group** is one logical subscriber. The partitions of a topic are divided between the members of the group, so adding instances spreads the load, up to one instance per partition. Different groups are independent: the notification service and the analytics pipeline each see every message, at their own pace. This is why the partition count is effectively your ceiling on parallelism for a single consumer, and why it's worth choosing it with some headroom. Increasing it later is possible but it changes which partition a key maps to, which breaks per-key ordering for messages in flight.

Kafka keeps messages for a retention period whether or not they have been consumed. That's a real difference from a classic queue. A consumer that was down for an hour simply resumes from its last committed offset, and a brand-new consumer can, if you want, read the whole history from the beginning.

2.3 What goes into an event

There are two schools here. **Thin events** carry an ID and little else (`{ orderId: "o-123" }`); consumers call back to the producer for details. **Fat events**, sometimes called event-carried state transfer, include everything a typical consumer needs: the customer's e-mail, the line items, the totals.

Thin events keep payloads small but bring back the synchronous coupling you were trying to get rid of, since every consumer now depends on the producer's API being up. We lean towards fat events, with one caveat: don't include data just because you have it. Every field you publish becomes part of a contract that's painful to change.

Whatever you choose, wrap every event in a common envelope. Ours looks like this:

```
{
  "id": "9b6c1f3e-...",
  "type": "order.placed.v1",
  "source": "order-api",
  "time": "2026-10-15T09:41:07.312Z",
  "subject": "o-123",
  "correlationId": "c-5d1e...",
  "data": { "customerId": "c-42", "email": "ann@example.com", "items": [ ... ] }
}
```

The `id` is what consumers use to spot duplicates. The `type` includes a version, so a breaking change becomes `order.placed.v2` and both can coexist on the topic while consumers migrate. The `subject` doubles as the Kafka key. The `correlationId` is the thread that ties together everything that happens because of one customer action, and it pays for itself the first time you have to debug a stuck order. If this shape looks familiar, it's because it borrows from the CloudEvents specification, which is worth reading even if you don't adopt it.

2.4 Schemas and evolution

Events outlive the code that produced them. A consumer reading from the beginning of a topic will see messages written by last year's version of the producer. That means you need a schema and a policy for changing it.

The policy can be simple. Adding an optional field is fine. Removing a field, renaming one, or changing its meaning is a new version. Consumers should ignore fields they don't understand. Producers should never assume a field is unused just because they can't think of who reads it.

For enforcement, teams tend to use either a schema registry (Confluent Schema Registry or AWS Glue Schema Registry, typically with Avro or Protobuf) or JSON Schema validated in code. We validate JSON on the way *in*, in every consumer. That catches bad producers early, and a message that can never pass validation is what's known as a poison message. It should go straight to a dead-letter topic rather than being retried forever (more on that below).

3. Choreography and orchestration

Once you have events, there are two ways to build a multi-step business process on top of them.

In **choreography**, each service reacts to events and emits its own. Inventory hears `OrderPlaced` and emits `StockReserved`; payment hears `StockReserved` and emits `PaymentSucceeded`; the order service hears that and marks the order confirmed. Nobody is in charge. It's loosely coupled and it's easy to add a new participant.

In **orchestration**, one component owns the process and tells the others what to do. An order saga sends `ReserveStock`, waits for the answer, sends `RequestPayment`, waits again, and decides whether to confirm or compensate.

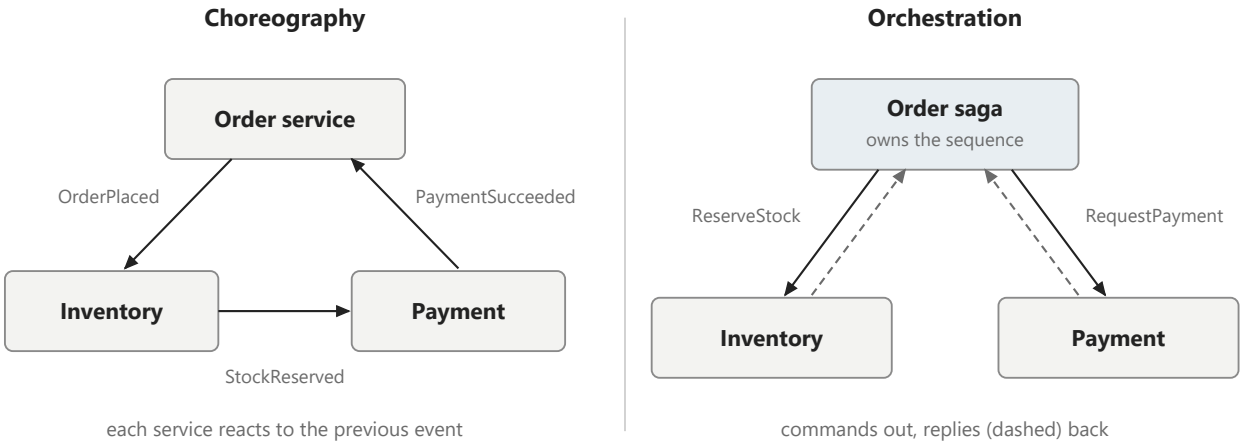


Figure 1. The same order flow, choreographed (left) and orchestrated (right).

Choreography is lovely for simple fan-out: an order was placed, so send an e-mail, update the search index, notify the warehouse. None of those steps depend on each other. It becomes a problem when the process has real logic in it, such as timeouts, compensation, or "do this only if that succeeded". At that point the logic is scattered across services, nobody can answer the question "what state is order o-123 in?" without reading four codebases, and changing the order of steps means coordinated deployments.

Orchestration puts the process in one place, which makes it readable and testable. The risk is that the orchestrator grows into a god service that knows too much about everyone else. The way to avoid that is to keep the orchestrator focused on the *sequence* (what happens next, what happens on failure) and leave the *how* to the services it talks to.

In practice most systems use both, and ours does. The order lifecycle (reserve, pay, confirm or compensate) is orchestrated by a Temporal workflow. Everything that merely reacts to the outcome (e-mails, the read model, analytics) is choreographed through Kafka. A reasonable test: if a step can fail in a way that requires undoing earlier steps, it belongs in an orchestrated process.

4. Living with at-least-once delivery

This section is the least glamorous in the document and probably the most important. Almost every production incident in an event-driven system comes down to a message being delivered twice, delivered out of order, or not delivered at all.

4.1 Why duplicates are normal

Kafka consumers commit offsets after processing a message. If the consumer crashes after doing the work but before committing, the message is delivered again when it restarts. Producers have the mirror-image problem: if the acknowledgement from the broker is lost, the producer retries and the message may be written twice. Kafka's idempotent producer fixes the second case for a single producer session, and Kafka transactions can give you exactly-once processing for pipelines that read from Kafka and write back to Kafka. But the moment your handler touches anything outside Kafka (a database, a payment provider, an SMTP server), you're back to at-least-once.

So the working assumption has to be: **every message may arrive more than once, and every handler must be safe to run more than once.** That property is called idempotency, and there are only a few ways to get it.

Some operations are naturally idempotent. Setting a status to `CONFIRMED` twice leaves it `CONFIRMED`. Others aren't: charging a card, incrementing a counter, sending an e-mail. For those you need to remember what you've already done.

4.2 The inbox

The **inbox** pattern records the ID of every event a consumer has processed, in the same database transaction as the handler's own changes. When an event arrives, the handler tries to insert its ID. If the insert fails because the ID is already there, the event is a duplicate and is skipped. If it succeeds, the handler does its work and both changes commit together.

The important detail is *the same transaction*. Recording the ID in Redis and the effect in a database leaves a gap where a crash produces either a lost event or a double effect. We'll see how to do this with DynamoDB a little later.

4.3 The outbox

The producing side has its own version of the problem. The order API needs to save the order *and* publish `OrderPlaced`. Those are two different systems and there's no transaction that spans both. Publish first and crash, and you've announced an order that doesn't exist. Save first and crash, and the order exists but nobody hears about it.

The **transactional outbox** solves this by writing the event into the same database as the order, in the same transaction, and having a separate relay publish it afterwards. The relay may publish an event twice (if it crashes before marking it as sent), which is fine, because consumers have inboxes. What it can't do is lose one.

The relay can be a polling loop, but there's a neater option when the database offers change data capture. With a relational database that's usually Debezium reading the write-ahead log. With DynamoDB it's DynamoDB Streams, which we'll get to.

4.4 Ordering

Kafka gives you order per partition, which, with the right key, means per aggregate. That's usually enough, but there are two ways to lose it without noticing.

The first is retries. If a consumer fails on message 5 and you push it to a retry topic to try again later, message 6 for the same key may be processed before it. Sometimes that's acceptable. For an order's lifecycle it usually isn't.

The second is the producer side. If two instances of a service publish events about the same order, Kafka orders them by arrival, not by when they happened. Funnelling all writes for an aggregate through one place (which is another thing the outbox gives you) avoids this.

When strict ordering matters and you can't guarantee it upstream, make consumers defensive: carry a version number or sequence on each event and ignore anything older than what you've already applied.

4.5 Dead letters

Some messages will fail no matter how many times you retry them: a payload that doesn't match the schema, a reference to a customer that was deleted, a bug in the handler. If the consumer keeps retrying, the partition is stuck and every message behind it waits.

The standard answer is a **dead-letter topic**. After a bounded number of retries with backoff (or immediately, for messages that fail validation) the original message is copied to `<topic>.dlq` with headers describing what went wrong, and the consumer moves on. Someone then looks at the DLQ, fixes the cause, and replays the messages.

Two things make DLQs useful rather than a place where messages go to be forgotten. Somebody has to be alerted when the DLQ isn't empty, and there has to be a tool, even a crude script, for replaying messages back to the original topic.

5. Temporal: durable workflows

Kafka is very good at moving facts around. It's not a good place to keep the state of a business process. If the order saga were built from Kafka consumers alone, you'd need a table for saga state, a scheduler for timeouts, logic for resuming after a crash, and a way to find out where each order is stuck. Teams build this all the time, and it tends to be the buggiest part of the system.

Temporal takes that whole category of problem away. You write the process as an ordinary async function, and Temporal makes it durable: every step's result is recorded, and if the worker running it dies, another worker picks it up and continues from exactly where it was. A `sleep('30 days')` really does sleep for thirty days, across deployments and restarts.

5.1 Workflows and activities

A Temporal application has two kinds of code. **Workflows** are the process: the sequence of steps, the branching, the waiting. **Activities** are the steps that touch the outside world: calling an API, writing to DynamoDB, publishing to Kafka. Activities can fail and are retried automatically according to a retry policy you control. Workflows don't fail because of infrastructure; they only fail if your logic decides they should.

Here's the core of our order saga:

```
import { proxyActivities, defineSignal, setHandler, condition } from '@temporalio/workflow';
import type * as activities from './activities';
import type { Order, PaymentOutcome } from './types';

const { reserveStock, releaseStock, requestPayment, confirmOrder, cancelOrder } =
  proxyActivities<typeof activities>({ startToCloseTimeout: '30 seconds' });

export const paymentResult = defineSignal<[PaymentOutcome]>('paymentResult');

export async function orderSaga(order: Order): Promise<string> {
  let payment: PaymentOutcome | undefined;
  setHandler(paymentResult, (outcome) => {
    payment ??= outcome; // the first answer wins; duplicates are ignored
  });

  await reserveStock(order);
  await requestPayment(order); // publishes a command to Kafka and returns

  const answered = await condition(() => payment !== undefined, '15 minutes');
  if (answered && payment?.ok) {
    await confirmOrder(order.id);
    return 'CONFIRMED';
  }

  await releaseStock(order.id); // compensation
  await cancelOrder(order.id, payment?.reason ?? 'Payment timed out');
  return 'CANCELLED';
}
```

It reads like the process description a product manager would write, and that's the point. The fifteen-minute wait is durable. If the payment service is down for ten minutes and the worker is redeployed twice in that time, the workflow is still waiting when the answer finally arrives.

5.2 How Temporal and Kafka meet

The two systems overlap a little, so it's worth being explicit about the division of labour in our design.

Kafka carries **facts between services**: an order was placed, a payment succeeded. Temporal holds **the state of processes**: this order is waiting for payment and will give up at 09:56. The connection between them is a pair of small bridge consumers.

The first listens for `OrderPlaced` and starts a saga, using a workflow ID derived from the order: `order-saga-o-123`. Temporal refuses to start a second workflow with the same ID, so a redelivered `OrderPlaced` can't create a duplicate saga. The bridge treats "already started" as success. That one decision makes the bridge idempotent for free.

The second listens for `PaymentSucceeded` and `PaymentFailed` and turns them into **signals** to the waiting workflow. Because the workflow keeps only the first answer, a duplicate signal is harmless. If the saga

has already finished (it timed out, say), the signal fails with "workflow not found". That's not an error for the bridge, though it *is* a business problem: money was taken for a cancelled order, and a real system would issue a refund there.

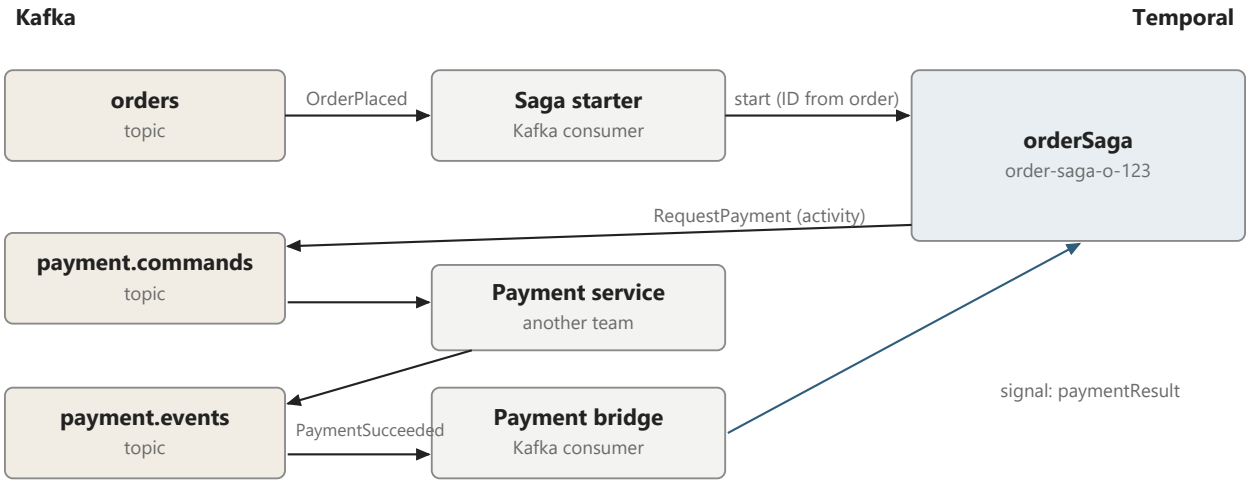


Figure 2. Two small bridges connect Kafka and Temporal: one starts sagas, the other turns payment events into signals.

Why not let the workflow call the payment service directly from an activity? You can, and for many services it's the simpler choice. We route payment through Kafka because the payment service is owned by another team, has its own scaling characteristics, and other consumers want its events too. Temporal doesn't force you to choose one style; activities can make HTTP calls, publish to Kafka, or both.

5.3 The rules you have to follow

Temporal's durability comes from replay. When a worker picks up a workflow, it re-runs the workflow code from the start and checks each step against the recorded history. For that to work, workflow code must be **deterministic**: given the same history, it must make the same decisions in the same order.

In practice that means no direct I/O, no `Math.random()` or `Date.now()` used for decisions (the TypeScript SDK replaces both with deterministic versions inside workflows), and no reading of environment variables or global state that might change. Anything non-deterministic goes in an activity. The TypeScript SDK helps by running workflows in an isolated sandbox where Node APIs aren't available.

The rule people trip over is versioning. If you change the order of steps in a workflow while executions are in flight, those executions will fail to replay. Temporal gives you `patched()` to branch between old and new code paths, and worker versioning to route old executions to old code. Both work, but you have to remember to use them. Keeping a replay test in CI (download a few real histories, replay them against the new code) catches most mistakes before they reach production.

5.4 Practical limits

A workflow's history isn't unlimited. Temporal warns at around 10,000 events and terminates a workflow at roughly 51,000 events or 50 MB of history. Individual payloads are limited to a couple of megabytes. None of that matters for an order saga with a dozen steps, but it does matter for a workflow that loops forever (a subscription that renews monthly, say). Those should call `continueAsNew` periodically to start a fresh history.

Keep payloads small in general. Pass IDs and the minimum data the workflow needs to make decisions, and let activities load the rest. A workflow that carries a full customer profile through every step will hit the limits sooner and is harder to evolve.

5.5 Running Temporal on AWS

You have two choices. Temporal Cloud is the managed service: you run only your workers, and the server, persistence and upgrades are someone else's problem. Self-hosting means running the Temporal server cluster yourself, usually on EKS, with a persistence store behind it. Supported stores are Cassandra, PostgreSQL and MySQL, so on AWS that typically means Aurora PostgreSQL. DynamoDB isn't a supported backend for the Temporal server itself; it's only your application's database.

For a small team, the managed service is usually the right call. Running the server well (scaling history shards, upgrades, monitoring the persistence layer) is a real job.

6. DynamoDB as the system of record

DynamoDB is a natural fit for event-driven services on AWS. It scales without much thought, it has no connection pools to exhaust when a hundred Lambda functions wake up at once, and its change stream gives you an outbox for almost nothing. It also punishes you quickly if you model data the way you would in a relational database.

6.1 Design for access patterns

In DynamoDB you design the table around the questions you'll ask, not around the entities you have. Every item has a partition key and usually a sort key, and efficient reads are those that go straight to one partition key. Scans exist but should be treated as a sign that the model is wrong.

For our order service the access patterns are few:

Access pattern	Key condition	Notes
Get an order with its lines	`pk = ORDER#o-123`	one query returns the order and its line items
Orders for a customer, newest first	GSI1: `gsi1pk = CUSTOMER#c-42`	sort key is the order date
Has this event been processed?	`pk = INBOX#payment-service`, `sk = <eventId>`	conditional put, with TTL
Events waiting to be published	`pk = OUTBOX#o-123`	written in the order's transaction, read through Streams

Putting different kinds of items in one table (single-table design) is common and lets one query return an order together with its line items. It isn't mandatory. If separate tables per entity make the service easier to understand, that's a legitimate choice, especially for small services that each own a handful of entities anyway.

Choose partition keys with high cardinality. A single partition can handle up to about 3,000 read units and 1,000 write units per second. Adaptive capacity helps with uneven traffic, but a key like `STATUS#PENDING` that every write touches will throttle no matter which capacity mode you're in.

6.2 Idempotency with conditional writes

DynamoDB's conditional writes are what make the inbox pattern cheap. A put with `attribute_not_exists(pk)` succeeds only if the item doesn't already exist. Wrap the inbox marker and the handler's own write in a transaction and you have exactly the "record it and do it, atomically" behaviour we wanted:

```
import { DynamoDBClient, TransactionCanceledException } from '@aws-sdk/client-dynamodb';
import { DynamoDBDocumentClient, TransactWriteCommand } from '@aws-sdk/lib-dynamodb';

const db = DynamoDBDocumentClient.from(new DynamoDBClient({}));
const TABLE = process.env.TABLE_NAME ?? 'commerce';
const WEEK = 7 * 24 * 3600;

// Returns false if this consumer has already processed the event.
export async function handleOnce(
  consumer: string,
  eventId: string,
  effect: Record<string, unknown>,
): Promise<boolean> {
  const expiresAt = Math.floor(Date.now() / 1000) + WEEK; // TTL attribute
  try {
    await db.send(new TransactWriteCommand({
      TransactItems: [
        {
          Put: {
            TableName: TABLE,
            Item: { pk: `INBOX#${consumer}`, sk: eventId, expiresAt },
            ConditionExpression: 'attribute_not_exists(pk)',
          },
        },
        { Put: { TableName: TABLE, Item: effect } },
      ],
    }));
    return true;
  } catch (err) {
    const duplicate =
      err instanceof TransactionCanceledException &&
      err.CancellationReasons?.[0]?.Code === 'ConditionalCheckFailed';
    if (duplicate) return false;
    throw err;
  }
}
```

The TTL attribute lets DynamoDB delete old markers for us. A week is plenty when Kafka retention is shorter than that; the marker only has to outlive any realistic redelivery. Note that TTL deletion is lazy and can lag by a day or two, so never rely on it for correctness, only for housekeeping.

Transactions can include up to 100 items and cost twice the write capacity of ordinary writes. Use them where atomicity matters, not by default.

6.3 DynamoDB Streams as the outbox

Every change to a table with Streams enabled shows up as a stream record, in order for each item, within about a second, and kept for 24 hours. That's most of an outbox relay already built.

The pattern we use: the order and an outbox item go into the table in one transaction. A Lambda function (or a small consumer using the Kinesis adapter) reads the stream, picks out new outbox items, and publishes their payload to Kafka. The outbox items carry a short TTL so they clean themselves up.

```
import { randomUUID } from 'node:crypto';
import { DynamoDBClient } from '@aws-sdk/client-dynamodb';
import { DynamoDBDocumentClient, TransactWriteCommand } from '@aws-sdk/lib-dynamodb';

const db = DynamoDBDocumentClient.from(new DynamoDBClient({}));
const TABLE = process.env.TABLE_NAME ?? 'commerce';

export async function placeOrder(
  order: { id: string; customerId: string },
  correlationId: string,
) {
  const event = {
    id: randomUUID(),
    type: 'order.placed.v1',
    source: 'order-api',
    time: new Date().toISOString(),
    subject: order.id,
    correlationId,
    data: order,
  };
  await db.send(new TransactWriteCommand({
    TransactItems: [
      {
        Put: {
          TableName: TABLE,
          Item: { pk: `ORDER#${order.id}`, sk: 'ORDER', status: 'PLACED', ...order },
          ConditionExpression: 'attribute_not_exists(pk)',
        },
      },
      {
        Put: {
          TableName: TABLE,
          Item: {
            pk: `OUTBOX#${order.id}`,
            sk: event.id,
            topic: 'orders',
            payload: JSON.stringify(event),
            expiresAt: Math.floor(Date.now() / 1000) + 24 * 3600,
          },
        },
      },
    ],
  }));
  return event;
}
```

Why not skip the outbox item and publish straight from the order's own stream records? You can, but then your public events are shaped by your table layout. Rename an attribute and every consumer breaks. An explicit outbox item keeps the event contract separate from the storage model.

A few details are worth knowing before you build this. Stream records are kept for 24 hours, so a relay that is broken for longer than that loses events. Alert on the iterator age long before that point. With

Lambda, configure a failure destination and bisect-on-error so one bad record doesn't block the shard. And since the relay can publish an event twice after a retry, the event `id` must be generated once, when the outbox item is written, and never at publish time.

There's also Kinesis Data Streams for DynamoDB, which offers longer retention and more consumers. The trade-off is that records can be duplicated or arrive out of order, so consumers have to sort it out themselves. For an outbox, plain DynamoDB Streams is usually the better fit.

6.4 Things that surprise people

Global secondary indexes are eventually consistent. A write to the table shows up in a GSI shortly after, not immediately, and you can't ask for a strongly consistent read from a GSI. Read-your-own-write flows should use the base table.

Items are limited to 400 KB. Large blobs belong in S3 with a pointer in the item.

On-demand capacity isn't unlimited. It scales quickly, but a sudden jump far above your previous peak can still throttle for a short while. Batch backfills and replays should be rate-limited on purpose.

Throttling looks like slowness. The AWS SDK retries throttled requests with backoff, so the first symptom is usually rising latency rather than errors. That's one reason the throttle metrics need monitors of their own.

7. Seeing what's happening: Datadog

In a synchronous system, a single trace shows you the whole request. In an event-driven one, a customer action becomes an HTTP request, a database write, a stream record, a Kafka message, a workflow, three activities, two more Kafka messages and an e-mail, some of which happen minutes apart. Observability is what keeps that from being a black box, and it has to be designed in. Adding it after the first incident is much harder.

Datadog covers the three things we need: traces that follow a message across services, metrics for Kafka, Temporal and DynamoDB in one place, and logs that link back to the trace they came from.

7.1 Tracing

The Node.js tracer, `dd-trace`, instruments HTTP servers and clients, the AWS SDK and the common Kafka clients automatically. It has to be loaded before anything else so it can patch those modules:

```
// tracer.ts: imported first, or run with: node --import dd-trace/initialize.mjs
import tracer from 'dd-trace';

tracer.init({
  service: 'order-api',
  env: process.env.DD_ENV,
  version: process.env.DD_VERSION,
  logInjection: true, // adds trace and span IDs to structured log lines
});

export default tracer;
```

Set `service`, `env` and `version` consistently on every service (Datadog calls this unified service tagging). It's what lets you filter a dashboard to "payment service, production, the version deployed an hour ago".

For Kafka, the tracer injects the trace context into message headers when producing and picks it up when consuming, so a consumer's span becomes a child of the producer's. Check that your client library is on the supported list. If it isn't, propagating the headers by hand with `tracer.inject` and `tracer.extract` is only a few lines of code.

Two things break the trace and need attention. DynamoDB Streams doesn't carry trace headers, so the hop from the order write to the outbox relay starts a new trace. Temporal isn't instrumented by `dd-trace` out of the box, but the SDK has an OpenTelemetry interceptor package (`@temporalio/interceptors-opentelemetry`), and the Datadog Agent accepts OpenTelemetry data. In both cases the correlation ID in the event envelope is the fallback. Tag spans and log lines with it, and you can always find every piece of one order's journey, even where the trace itself is split:

```
import tracer from 'dd-trace';

export function tagOrder(orderId: string, correlationId: string) {
  const span = tracer.scope().active();
  span?.setTag('order.id', orderId);
  span?.setTag('correlation.id', correlationId);
}
```

7.2 Data Streams Monitoring

Traces are good for following a single order. They're poor at answering questions about the pipeline as a whole: which consumers are falling behind, how long messages take end to end, where the throughput dropped. Datadog's Data Streams Monitoring is built for that. Enable it with `DD_DATA_STREAMS_ENABLED=true` on producers and consumers, and it maps the pathways between services and topics and shows latency and lag along each one.

The metric that matters most is **consumer lag**, ideally measured in time rather than in messages. "Ten thousand messages behind" means nothing on its own; "four minutes behind" tells you whether customers are noticing.

7.3 Temporal metrics

Temporal workers emit metrics that tell you whether the workers are keeping up. In the TypeScript SDK you expose them in Prometheus format once, when the process starts:

```
import { Runtime } from '@temporalio/worker';

Runtime.install({
  telemetryOptions: {
    metrics: {
      prometheus: { bindAddress: '0.0.0.0:9464' },
    },
  },
});
```

The Datadog Agent then scrapes the endpoint with its OpenMetrics check:

```
init_config:
instances:
  - openmetrics_endpoint: http://localhost:9464/metrics
    namespace: temporal_worker
    metrics:
      - temporal_.*
```

The two numbers to watch are the **schedule-to-start latencies** for workflow tasks and for activity tasks. They measure how long work sits in a task queue before a worker picks it up. If they climb, you don't have enough workers (or enough slots per worker), and everything downstream slows down with no errors anywhere. Beyond those, alert on workflow failures and on activity failures that exceed their usual rate.

If you use Temporal Cloud, Datadog has an integration that pulls the cloud-side metrics as well. For a self-hosted cluster, there's an integration for the server's own metrics.

7.4 DynamoDB metrics

DynamoDB's CloudWatch metrics arrive through the Datadog AWS integration. The useful ones are the throttle events (`ReadThrottleEvents`, `WriteThrottleEvents`), consumed capacity relative to what's provisioned, `SystemErrors`, and for Streams consumers the iterator age (`IteratorAge` on the Lambda function that reads the stream). Iterator age is the outbox relay's version of consumer lag, and with a 24-hour stream retention it's the one you least want to miss.

7.5 Monitors that are worth having

Dashboards are for investigating. Monitors are what wake people up, and there should be few of them, each tied to something a customer would notice. Ours, roughly in order of importance:

- Consumer lag (in seconds) above a threshold for any business-critical consumer group.
- Any message in any dead-letter topic.
- DynamoDB Streams iterator age above fifteen minutes.
- Temporal activity schedule-to-start latency above a few seconds for more than five minutes.
- Orders stuck in `PLACED` longer than the saga's own timeout, which catches everything the others miss.
- Error rate and latency on the order API, which is still the front door.

The last-but-one deserves a comment. It's a business-level check: count orders older than twenty minutes that haven't reached a final state. Technical monitors tell you a component is unhealthy. This one tells you a customer is affected, and it catches failure modes nobody thought to write a monitor for.

8. Putting it together

Here is the whole platform on AWS, with each piece where it ends up in our deployment:

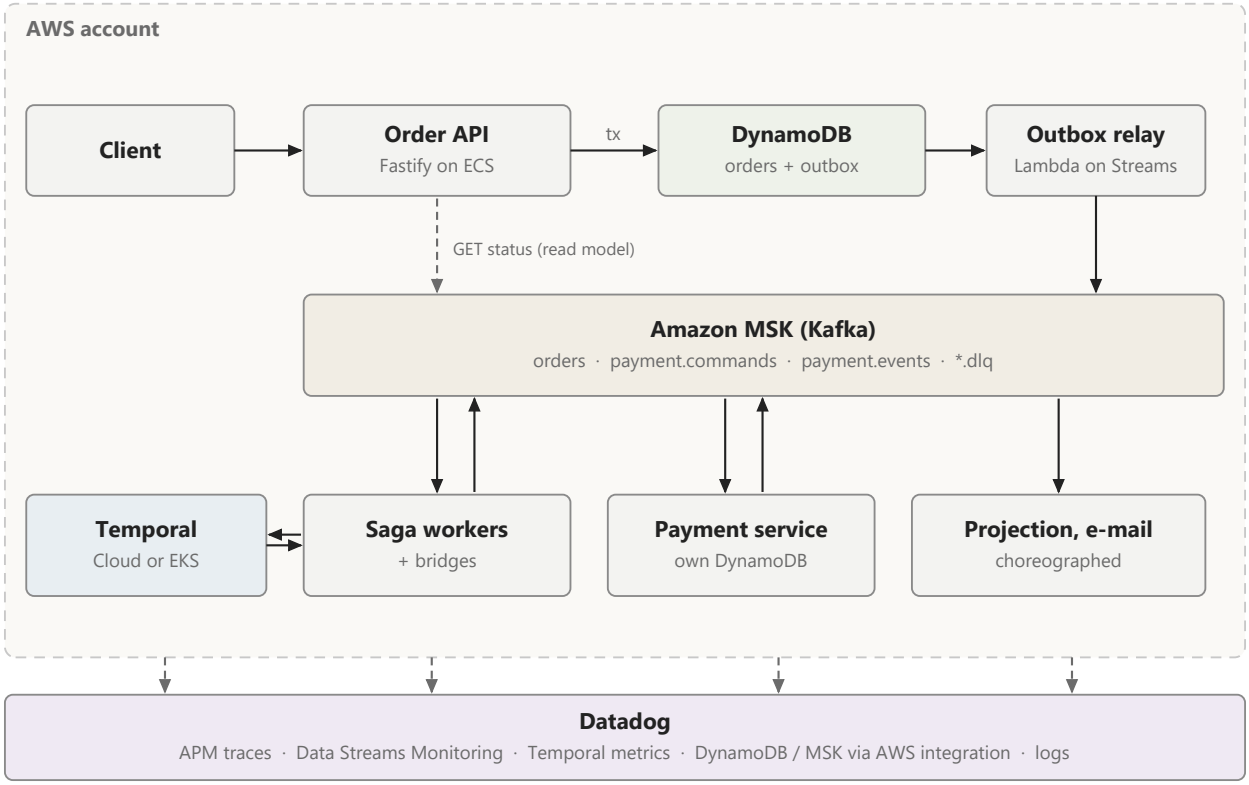


Figure 3. Reference deployment on AWS. Dashed arrows into Datadog are agents and integrations, not application traffic.

The order API is a Fastify service on ECS Fargate behind a load balancer. It writes the order and an outbox item to DynamoDB in one transaction and answers `202 Accepted` with a link to the order's status. It doesn't talk to Kafka at all.

The outbox relay is a Lambda function attached to the table's stream. It publishes new outbox items to the `orders` topic on Amazon MSK, keyed by order ID.

The saga starter consumes `orders`, and for each `OrderPlaced` starts an `orderSaga` workflow on Temporal with the ID `order-saga-<orderId>`. The saga reserves stock through an activity, publishes `RequestPayment` to the payment team's command topic, and waits.

The payment service consumes its command topic, charges the card (using its own DynamoDB inbox to make sure it charges once), and publishes `PaymentSucceeded` or `PaymentFailed`. The payment bridge turns that into a signal, and the saga confirms the order or compensates.

Meanwhile, the order-status projection and the notification service consume `orders` independently. The projection keeps a read model that the API's `GET` endpoint serves; the notifier sends the e-mails. Neither is known to the saga or to each other.

Every service runs the Datadog Agent as a sidecar (or the Lambda extension, for the relay), with unified service tags and Data Streams Monitoring enabled.

8.1 The life of one order

It's worth walking through a single order with timings, because it makes the asynchronous nature concrete.

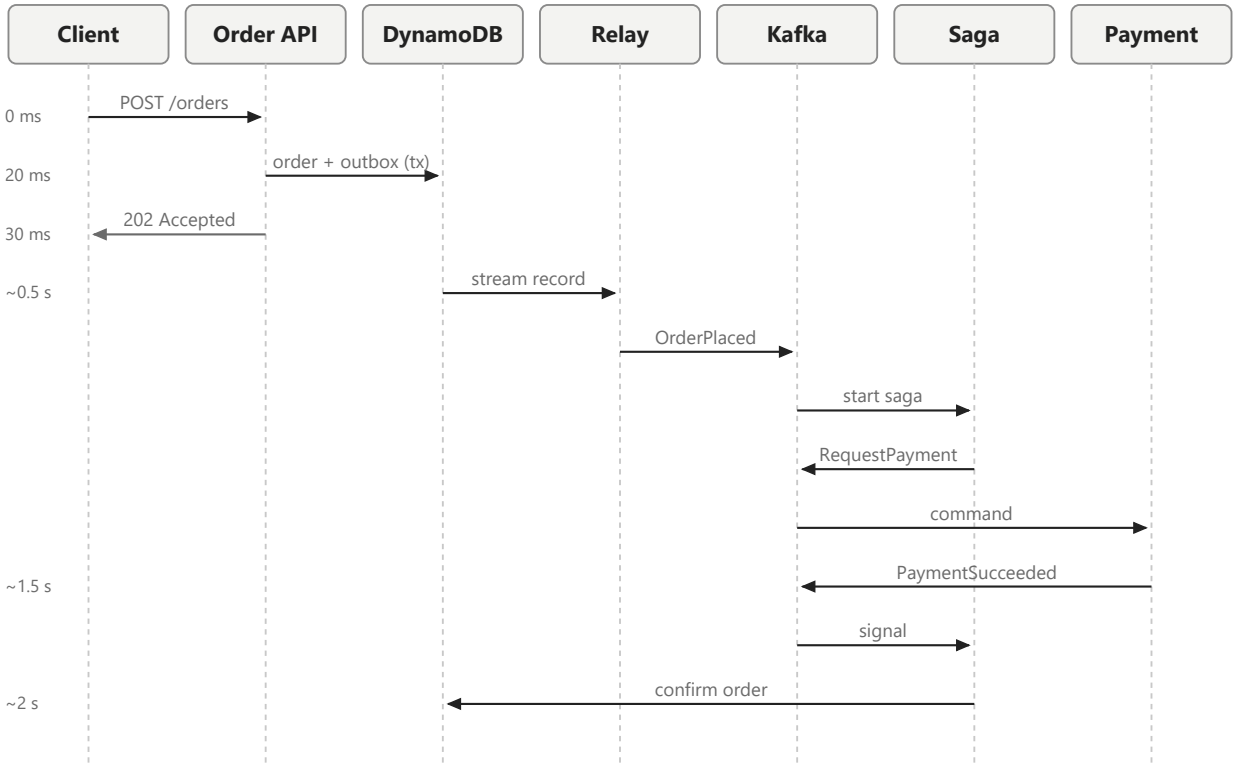


Figure 4. One order on a good day. The client has its answer after 30 ms; the rest happens asynchronously in about two seconds.

At 0 ms the API accepts the request. By 30 ms the transaction has committed and the client has its `202`. Somewhere between 100 ms and a second later, the stream record reaches the relay and `OrderPlaced` is on Kafka. A few milliseconds after that the saga starts, reserves stock and sends the payment command. The payment service usually answers within a second or two, and the order is confirmed. The customer sees "confirmed" on their next poll, or immediately if you push status updates over a WebSocket or server-sent events.

On a bad day the payment service is down. Nothing in that timeline fails. The command sits in Kafka, the saga waits durably, and the order shows "processing". When payment comes back, it works through the backlog and the orders confirm. If it stays down past the fifteen-minute deadline, the sagas release the stock, cancel the orders, and the customers get an e-mail. Whichever way it goes, every order ends in a defined state, which is the whole point.

9. Running it

Here are the problems we'd expect to see in the first months of running a system like this, and where to start looking.

Symptom	Likely cause	First thing to check
Orders stuck in PLACED, no errors anywhere	Outbox relay not running or stream iterator stuck	Lambda iterator age and errors for the stream consumer
Orders stuck in PLACED, saga exists	Workers not picking up tasks	Temporal schedule-to-start latency, worker count and health
Customer charged twice	A handler without an inbox, or an event ID generated at publish time	The payment service's inbox records for that order
Consumer lag grows steadily	Too few consumers for the partition count, or a slow dependency	Per-partition lag, handler latency in APM
Lag on one partition only	A hot key, or one poison message being retried	DLQ topic, the oldest unprocessed offset on that partition
Workflow fails after a deploy with a non-determinism error	Workflow code changed without `patched()`	The replay test; the diff of the workflow file
Latency up on DynamoDB calls, no errors	Throttling absorbed by SDK retries	Throttle events for the table and its GSIs

A few habits help more than any particular tool.

Make replays boring. Sooner or later you'll need to re-publish a day's events or replay a DLQ. If every consumer is idempotent, that's a routine operation. If some aren't, it's an incident.

Rehearse the failure modes. Stop the payment service in a staging environment and watch what happens. Kill a worker in the middle of a saga. Publish a malformed message. Each of these takes ten minutes and tells you more than a week of reading code.

Keep the correlation ID everywhere. In events, in log lines, in span tags, in the Temporal workflow's search attributes. The first question in every investigation is "what happened to this order?", and it should take one search to answer.

Write down who owns each topic. Every topic needs an owner who decides on its schema, its retention and its partition count. Topics without owners collect consumers nobody knows about, and then nobody dares change them.

10. Before going live

A short list to go through with the team before the first real order:

- Every consumer handles duplicates, and there's a test that proves it.
- Every topic has a key chosen for ordering, an owner, and a dead-letter topic.
- Event schemas are versioned and validated on the way in.
- The outbox relay's lag is monitored, with an alert well inside the 24-hour stream retention.
- Workflows have replay tests in CI, and everyone knows when to use `patched()`.
- Activity retry policies and timeouts are set deliberately, not left at defaults.
- Every saga has a deadline and a compensation path, and both are tested.
- Correlation IDs flow from the API through every event, workflow and log line.
- Monitors exist for consumer lag, DLQ depth, iterator age, schedule-to-start latency, and stuck orders.
- There's a documented way to replay a DLQ, and someone has done it at least once.

Further reading

- Martin Kleppmann, *Designing Data-Intensive Applications*. Chapters 11 and 12 on stream processing remain the best background on logs, ordering and derived data.
- Chris Richardson, *Microservices Patterns*. The chapters on sagas and on transactional messaging cover the outbox and the inbox in more depth.
- Alex DeBrie, *The DynamoDB Book*, for data modelling around access patterns.
- The Temporal documentation, in particular the pages on workflow determinism, versioning and worker performance.
- Datadog's documentation for APM in Node.js and for Data Streams Monitoring.
- The CloudEvents specification, for a well-thought-out event envelope.
- The *eda-commerce* repository (github.com/tcsr/eda-commerce): a runnable version of the order flow described here, using Kafka, Temporal and Fastify.